# CASE STUDY 10

## BUS PASSENGER OCCUPANCY SIMULATION

*Python Programming – Activity-Based Learning*

### 1. Problem Statement

Create a simple Python program to track and manage the number of passengers in a bus across different stops. The program should take the count of passengers boarding and getting off at each stop, update the total headcount, and make sure it never goes over the total bus capacity or falls below zero.

### 2. Objectives

- To learn how to take user inputs inside a loop.
- To use standard if-elif-else statements for conditions.
- To practice basic addition and subtraction counters.
- To display clean, real-time updates after each stop.

### 3. Assumptions

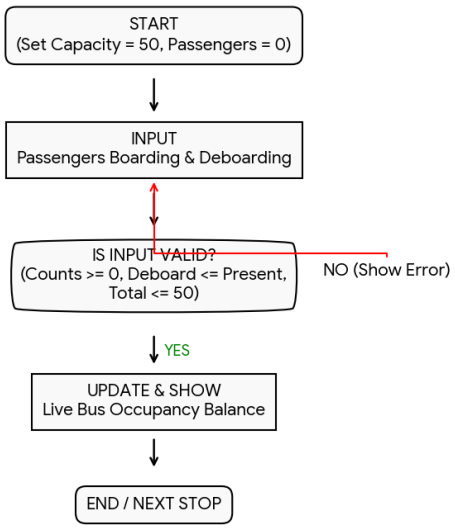
- The maximum capacity of the bus is fixed at 50 seats.
- The bus starts completely empty (0 passengers).
- Passenger counts cannot be negative numbers.
- Passengers getting off cannot be more than the current people on the bus.

### 4. Algorithm

1. Start the program and set capacity to 50, current passengers to 0.
2. Take input for passengers boarding and passengers getting off.
3. Check if inputs are negative or invalid; show error if true.
4. Check if numbers cross the max limit or drop below zero; show error if true.
5. Calculate the new total and print the live status.
6. Stop when the user chooses to close the route.

### 5. Flowchart

*Simplified Student-Style Flowchart Matrix (Reduced Blocks):*


START
(Set Capacity = 50, Passengers = 0)
INPUT
Passengers Boarding & Deboarding
IS INPUT VALID?
(Counts >= 0, Deboard <= Present,
Total <= 50)
NO (Show Error)
YES
UPDATE & SHOW
Live Bus Occupancy Balance
END / NEXT STOP

## 6. Pseudocode

```
START
    SET capacity = 50
    SET current = 0

    WHILE True
        INPUT board, deboard

        IF board < 0 OR deboard < 0 THEN
            PRINT "Error: Invalid numbers"
        ELSE IF deboard > current THEN
            PRINT "Error: Not enough passengers on board"
        ELSE IF (current - deboard + board) > capacity THEN
            PRINT "Error: Cannot exceed 50 seats"
        ELSE
            current = current - deboard + board
            PRINT "Current Passengers: ", current
        END IF
    END WHILE
STOP
```

## 7. Python Source Code

```
# Case Study 10: Bus Occupancy Simulator
capacity = 50
current = 0

print("--- Bus Route Simulation Started ---")

while True:
    choice = input("\nEnter 's' for next stop or 'e' to exit: ").lower()
    if choice == 'e':
        print("Route completed successfully.")
        break
    elif choice == 's':
        board = int(input("Enter passengers entering: "))
        deboard = int(input("Enter passengers leaving: "))

        if board < 0 or deboard < 0:
            print("Invalid input! Numbers cannot be negative.")
        elif deboard > current:
            print("Invalid input! More passengers cannot leave than present.")
        elif (current - deboard + board) > capacity:
            print("Overload error! Total cannot cross 50 seats.")
        else:
            current = current - deboard + board
            print("Stop Updated. Live Count:", current, "/", capacity)
    else:
        print("Please enter a valid choice ('s' or 'e').")
```

## 8. Test Cases

| Test ID | Current Count | Boarding | Deboarding | Expected Output Message | Result |
|---|---|---|---|---|---|
| TC_01 | 0 | 20 | 0 | Stop Updated. Live Count: 20 / 50 | Pass |
| TC_02 | 20 | 10 | 5 | Stop Updated. Live Count: 25 / 50 | Pass |
| TC_03 | 25 | 0 | 30 | Invalid input! More passengers cannot leave... | Pass |
| TC_04 | 25 | 35 | 0 | Overload error! Total cannot cross 50 seats. | Pass |

## 9. Sample Output Screenshots

```
--- Bus Route Simulation Started ---

Enter 's' for next stop or 'e' to exit: s
Enter passengers entering: 20
Enter passengers leaving: 0
Stop Updated. Live Count: 20 / 50

Enter 's' for next stop or 'e' to exit: s
Enter passengers entering: 40
Enter passengers leaving: 5
Overload error! Total cannot cross 50 seats.
```

## 10. Conclusion

This assignment helped me understand how to control loops and dynamic variable updates in Python using basic mathematical logic. By applying standard validations, the code handles standard errors perfectly without crashing during execution.